Lab 8: KNN Classifier Implementation & Hyperparameter Analysis

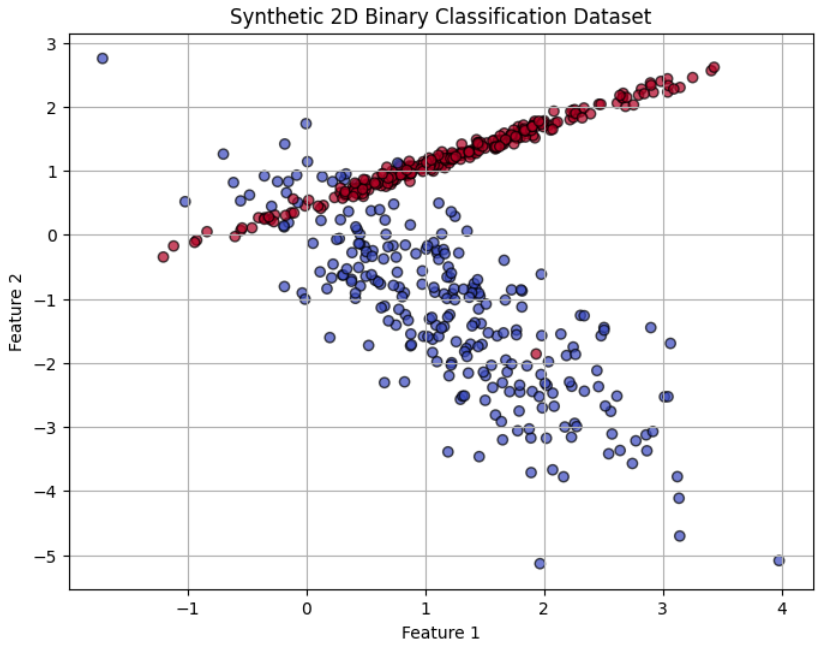
We will implement a K-Nearest Neighbors (KNN) classifier using `scikit-learn` on a synthetic classification dataset and evaluate the performance of different K values and distance metrics (Euclidean vs. Manhattan).

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Generate a synthetic 2D dataset for easy visualization
X, y = make_classification(
    n_samples=500,
    n_features=2,
    n_redundant=0,
    n_informative=2,
    random_state=42,
    n_clusters_per_class=1,
    class_sep=1.2
)

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Plot the dataset
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='coolwarm', edgecolors='k', alpha=0.7)
plt.title("Synthetic 2D Binary Classification Dataset")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid(True)
plt.show()
```



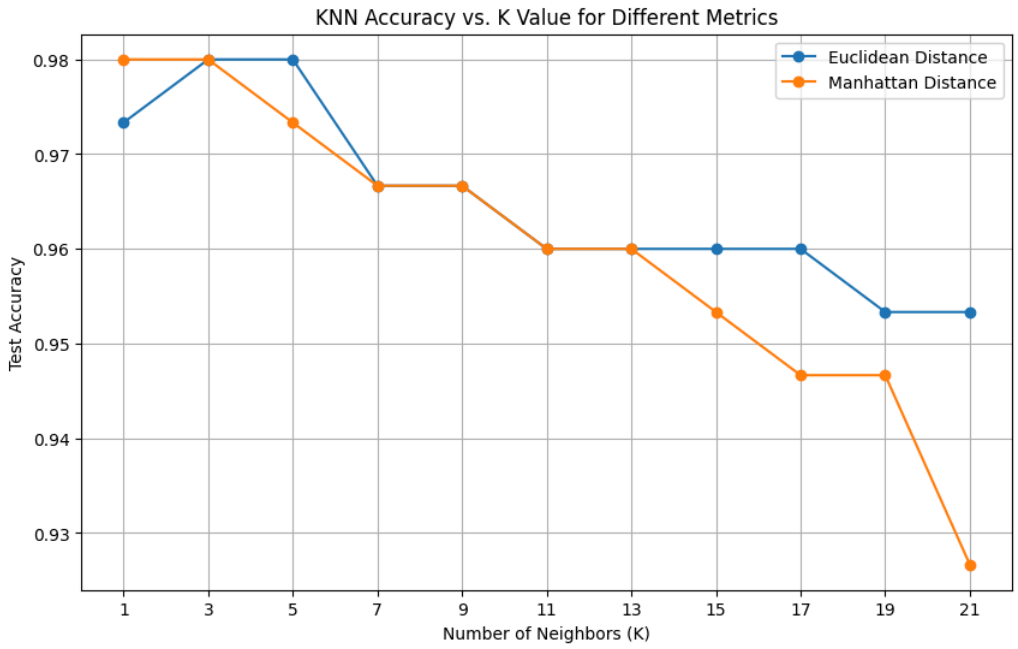

```
# 2. Analyze the impact of different K values and distance metrics
k_values = list(range(1, 22, 2))
metrics = ['euclidean', 'manhattan']
results = []

for metric in metrics:
    accuracies = []
    for k in k_values:
        knn = KNeighborsClassifier(n_neighbors=k, metric=metric)
        knn.fit(X_train, y_train)
        y_pred = knn.predict(X_test)
        acc = accuracy_score(y_test, y_pred)
        accuracies.append(acc)
    results.append((metric, accuracies))

# Plot the performance comparison
plt.figure(figsize=(10, 6))
for metric, accuracies in results:
    plt.plot(k_values, accuracies, marker='o', label=f'{metric.capitalize()} Distance')

plt.title('KNN Accuracy vs. K Value for Different Metrics')
plt.xlabel('Number of Neighbors (K)')
```

```
plt.ylabel('Test Accuracy')
plt.xticks(k_values)
plt.legend()
plt.grid(True)
plt.show()
```



Lab 10: K-Means Clustering & Cluster Optimization

We will perform clustering using the K-Means algorithm on an unsupervised dataset. To determine the optimal number of clusters, we will apply:

1. **The Elbow Method** (Within-Cluster Sum of Squares)
2. **The Silhouette Score**

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score

# 1. Generate synthetic clustering data
X_clust, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=42)

# Visualize raw data before clustering
plt.figure(figsize=(6, 4))
plt.scatter(X_clust[:, 0], X_clust[:, 1], s=30, color='gray')
plt.title("Raw Clustering Dataset")
plt.grid(True)
plt.show()
```




```
# 2. Determine optimal clusters using Elbow Method and Silhouette Score
wcss = []
silhouette_coefficients = []
K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, init='k-means++', max_iter=300, n_init=10, random_state=42)
    kmeans.fit(X_clust)
```

```
    wcss.append(kmeans.inertia_)

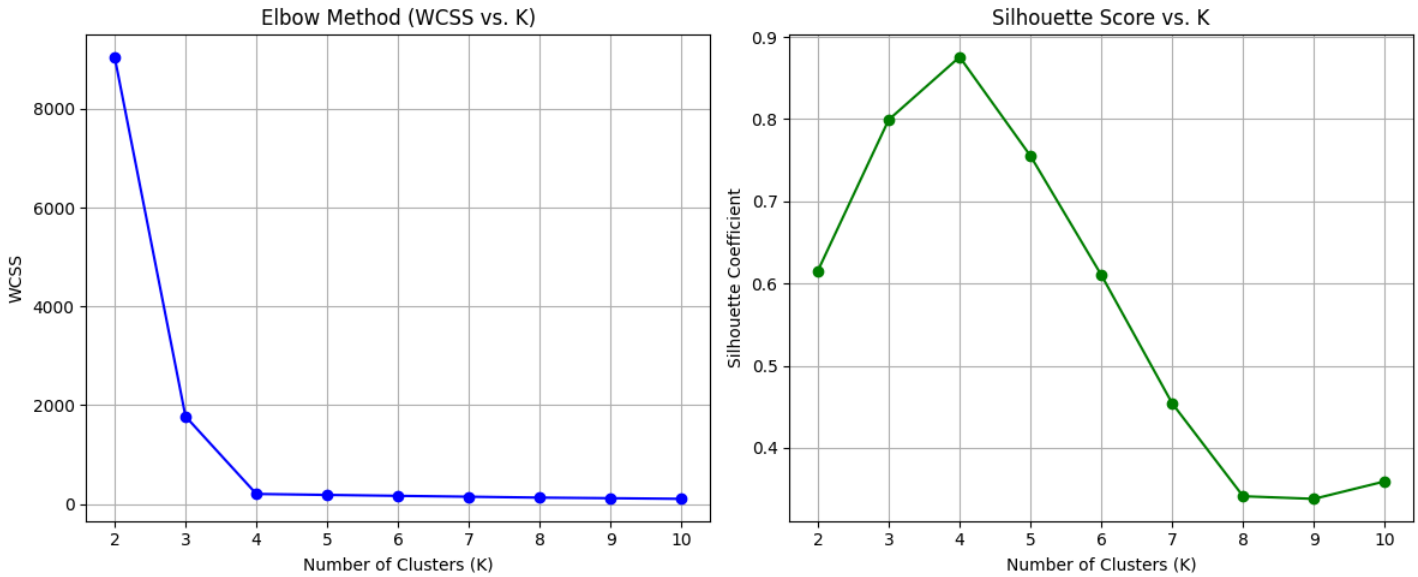
    score = silhouette_score(X_clust, kmeans.labels_)
    silhouette_coefficients.append(score)

# Plot the Elbow Method
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(range(2, 11), wcss, marker='o', color='blue')
plt.title('Elbow Method (WCSS vs. K)')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.grid(True)

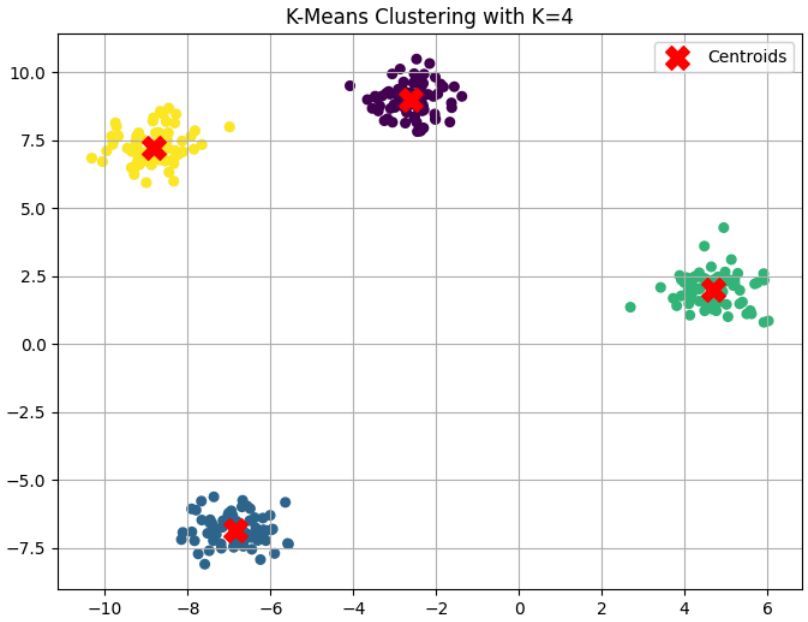
# Plot the Silhouette Scores
plt.subplot(1, 2, 2)
plt.plot(K_range, silhouette_coefficients, marker='o', color='green')
plt.title('Silhouette Score vs. K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Silhouette Coefficient')
plt.grid(True)

plt.tight_layout()
plt.show()
```




```
# 3. Fit optimal K-Means (e.g., K=4 based on evaluation) and visualize the outcome
optimal_k = 4
kmeans_opt = KMeans(n_clusters=optimal_k, init='k-means++', max_iter=300, n_init=10, random_state=42)
y_kmeans = kmeans_opt.fit_predict(X_clust)

plt.figure(figsize=(8, 6))
plt.scatter(X_clust[:, 0], X_clust[:, 1], c=y_kmeans, s=30, cmap='viridis', zorder=1)
plt.scatter(kmeans_opt.cluster_centers_[:, 0], kmeans_opt.cluster_centers_[:, 1], s=200, c='red', marker='X', label='Centroids', zorder=2)
plt.title(f"K-Means Clustering with K={optimal_k}")
plt.legend()
plt.grid(True)
plt.show()
```



Lab 9: Model Evaluation & Hyperparameter Tuning

We will evaluate model performance using comprehensive metrics and search strategies:

- **Metrics**: Accuracy, Precision, Recall, F1-Score, and ROC-AUC
- **Resampling**: Cross-validation
- **Hyperparameter Search**: Grid Search & Random Search

```
from sklearn.metrics import classification_report, roc_auc_score, roc_curve
from sklearn.model_selection import cross_val_score, GridSearchCV, RandomizedSearchCV

# 1. Basic model evaluation using various classification metrics
best_knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
best_knn.fit(X_train, y_train)
y_pred = best_knn.predict(X_test)
y_probs = best_knn.predict_proba(X_test)[:, 1]

print("--- Classification Report ---")
print(classification_report(y_test, y_pred))

roc_auc = roc_auc_score(y_test, y_probs)
print(f"ROC-AUC Score: {roc_auc:.4f}")

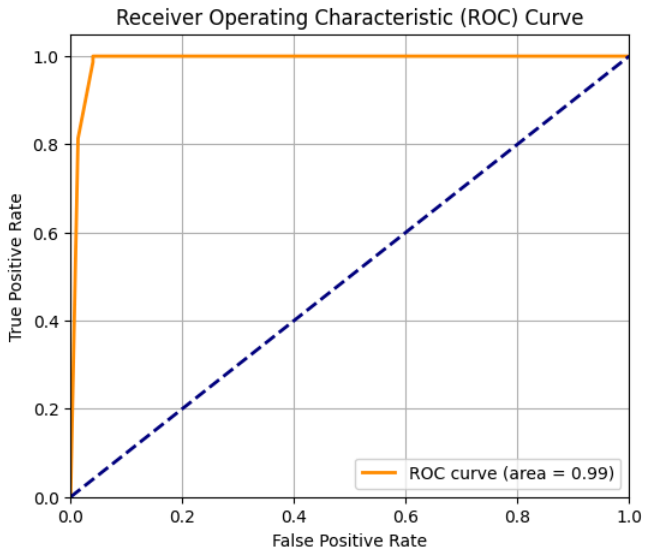
# Plot ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_probs)
plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

```
--- Classification Report ---
              precision    recall  f1-score   support

           0       1.00      0.96      0.98        75
           1       0.96      1.00      0.98        75

    accuracy                           0.98       150
   macro avg       0.98      0.98      0.98       150
weighted avg       0.98      0.98      0.98       150

ROC-AUC Score: 0.9894
```




```
# 2. Cross-validation
```